📝 Lecture 8

---

1. Polymorphism

Definition

- **Polymorphism** = “many forms”
- Enables a **single action** to behave differently for different objects

---

Types of Polymorphism

🧩 Compile-time Polymorphism (Static Polymorphism)

- Achieved via **method overloading**
- Methods have the **same name but different parameters**
- Can be applied to **both instance and static methods**

⚙️ Runtime Polymorphism (Dynamic Polymorphism)

- Achieved via **method overriding**
- Determined **at runtime**
- Requires **inheritance**
- **Data members cannot be overridden** → applies only to methods

---

2. Runtime Polymorphism

2.1 Dynamic Method Dispatch

- Occurs when a **superclass reference variable** refers to a **subclass object** (upcasting).
- The method that gets executed is determined by the **actual object type at runtime**, not the reference type.
- This works only for **overridden methods**.

Example

```
class Parent {
    void show() { System.out.println("Parent"); }
}

class Child extends Parent {
    @Override
    void show() { System.out.println("Child"); }
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child(); // upcasting
```

```
        p.show(); // Output: Child
    }
}
```

2.2 Binding

1. Field binding

- **Field access is compile-time bound**
- **No polymorphism for fields.**
- **just based on reference type**

```
class Parent { int x = 10; }
class Child extends Parent { int x = 20; }

Parent p = new Child();
System.out.println(p.x); // Output: 10 → field access based on reference type
```

2. Method Binding

- Method calls are **dynamically bound (runtime binding)** when the method is overridden
- Determined by the **actual object type**
- This is the basis of **runtime polymorphism**

```
class Parent {
    void show() { System.out.println("Parent"); }
}

class Child extends Parent {
    @Override
    void show() { System.out.println("Child"); }
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.show(); // prints "Child"
    }
}
```

Binding Type	When Determined	Notes
Static Binding (Early Binding)	Compile-time	field access, private method, final method, and static methods
Dynamic Binding (Late Binding)	Runtime	Overridden instance methods

---

4. `instanceof` Operator

- Used to check whether an object is an **instance of a class, subclass, or interface**
- Returns a **boolean value (true or false)**
- If the object is `null`, it always returns **false**

Example

```
Parent p = new Child();

System.out.println(p instanceof Child);   // true
System.out.println(p instanceof Parent);  // true
System.out.println(null instanceof Parent); // false
```

5. Upcasting vs Downcasting

Upcasting

- Upcasting is casting a **subclass object to a superclass reference**
- It is **always safe**
- Done **implicitly (no explicit cast required)**
- Used for **runtime polymorphism**

```
class Parent {
    void show() { System.out.println("Parent show"); }
}

class Child extends Parent {
    @Override
    void show() { System.out.println("Child show"); }

    void play() { System.out.println("Child play"); }
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child(); // Upcasting (automatic)
        p.show(); // Child show → runtime polymorphism

        // p.play(); ❌ Not accessible (reference type rule)
    }
}
```

Downcasting

- Downcasting is casting a **superclass reference to a subclass reference**
- Requires **explicit casting**
- Only safe if the actual object is truly of the target type
- Otherwise, it throws a **ClassCastException at runtime**

Example

```
// ❌ Unsafe downcasting (runtime error)
Parent p2 = new Parent();
Child c2 = (Child) p2; // ClassCastException

// ✅ Safe downcasting
Parent p = new Child();

if (p instanceof Child) {
    Child c = (Child) p;
    c.play();
}}
```

🧠 Memory Representation (Upcasting / Downcasting)

```
Heap:
[ Child object ]
 ├── Parent fields
 ├── Child fields
 └── class pointer → Child.class (Method Area)

Reference variable:
p ---> points to Child object
c ---> points to same Child object
```

🔥 Key Idea

- Both `p` and `c` point to the **same object in heap**
- Only the **reference type changes**, not the object
- Object always remains **Child**

---

6. Access Modifiers (default and protected)

1️⃣ Default Access(Package-Private Access)

- No modifier written (just leave it blank)
- Accessible **only within the same package**
- Not accessible from subclasses in **different packages**

Example

File: `package1/Parent.java`

```
package package1;

class Parent {           // default access
    int data = 100;      // default access
}
```

File: `package1/Main.java`

```
package package1;

public class Main {
    public static void main(String[] args) {
        Parent p = new Parent();
        System.out.println(p.data); // ✅ Accessible (same package)
    }
}
```

File: `package2/Other.java`

```
package package2;
import package1.Parent;

public class Other {
    public static void main(String[] args) {
        Parent p = new Parent();
        System.out.println(p.data); // ❌ Compile-time error (different package)
```

```
    }
}
```

2️⃣ Protected Access

- Accessible **within the same package** (like default)
- Also accessible in **subclasses**, even in **different packages**

Example

File: `package1/Parent.java`

```
package package1;

public class Parent {
    protected int data = 100;
}
```

File: `package2/Child.java`

```
package package2;
import package1.Parent;

public class Child extends Parent {
    public void showData() {
        System.out.println(data); // ✅ Accessible (protected)
    }
}
```

File: `package2/Child.java`

```
public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.showData(); // Output: 100
    }
}
```